

Bayesian Machine Learning in Quantitative Finance

Chapter 10: Bayesian Neural Networks for Motor Insurance Claims

Based on Mongwe, Mbuva & Marwala

Springer, 2025

Chapter Overview

- 1 10.1 Introduction
- 2 10.2 Bayesian Neural Network Models
- 3 10.3 Laplace Approximation to Bayesian Neural Networks
- 4 Toy Running Example: A Tiny BNN for Motor Claims
- 5 10.4 Numerical Experiments
- 6 10.5 Results and Discussions
- 7 10.6 Conclusion

Outline

- 1 10.1 Introduction
- 2 10.2 Bayesian Neural Network Models
- 3 10.3 Laplace Approximation to Bayesian Neural Networks
- 4 Toy Running Example: A Tiny BNN for Motor Claims
- 5 10.4 Numerical Experiments
- 6 10.5 Results and Discussions
- 7 10.6 Conclusion

10.1 Introduction: Motor Insurance and the Claim Prediction Problem I

Insurance is a form of risk management: a policyholder pays a premium so that, if something bad happens, the insurer covers the loss. **Motor insurance** is the subset of general (non-life) insurance concerned with vehicles — comprehensive cover typically includes theft, accidents, and third-party liability. Because motor insurance generates enormous amounts of policy and claims data, it is a natural target for machine learning.

Why Claim Prediction Matters

For every policy, an insurer wants to know: *how likely is this policyholder to file a claim this year?* Answering this well matters for two reasons: (1) **pricing** – premiums should reflect risk, and (2) **fraud detection** – unusual claim patterns can be flagged. Traditionally, insurers used **Generalised Linear Models (GLMs)** that assume a linear relationship between policy features and claim frequency/severity, and assume claim frequency and severity are independent. Real claims data rarely obey either assumption exactly.

10.1 Introduction: Motor Insurance and the Claim Prediction Problem II

The Shift Toward Machine Learning

As computing power and data volumes have grown, machine learning models – gradient-boosted trees and neural networks in particular – have increasingly been used to model claim frequency and severity, since they can capture non-linear relationships and interactions between features that a GLM would miss. Neural networks in particular have repeatedly been found to be strong predictors of claim size and claim occurrence in the actuarial literature.

The Gap This Chapter Fills

Almost all of this machine learning literature produces a single **point prediction**: “this policy has an 8% chance of a claim,” full stop. Very little attention has been paid to **Bayesian** approaches, which instead return a *whole distribution* over that probability – telling you not just the estimate, but how confident the model is in that estimate. This chapter fills that gap: it uses **Bayesian Neural Networks (BNNs)** with **Automatic Relevance Determination (ARD)** priors to predict whether a motor insurance policy will generate a claim, trained via the **Laplace approximation**. The ARD prior has a second benefit: it lets us rank which input features matter most for the prediction.

10.1 Introduction: Motor Insurance and the Claim Prediction Problem III

Two Uses of Bayesian Inference in this Chapter

- **Parameter estimation:** given a fixed network architecture, find the plausible values (really, the full *distribution* of plausible values) for its weights. This is usually done with sampling methods such as Markov chain Monte Carlo (MCMC).
- **Model selection:** compare *different* architectures (e.g., a small network vs. a deep one) using the **Bayesian evidence** $P(D | H)$ – the probability of the observed data D under model/architecture H , averaged over all possible weight settings. Higher evidence means the architecture explains the data better *without* needing to fit held-out data first.

Evaluating the evidence exactly is hard for a model as complex as a neural network, so approximations are needed: Bayesian quadrature, nested sampling, and the **Laplace approximation** of MacKay (1992) are the three most established options. MCMC is guaranteed to converge to the true posterior eventually but is computationally expensive; this chapter instead uses the Laplace approximation, which is far cheaper and, as we will see, scales comfortably to networks with hundreds of parameters.

10.1 Introduction: Motor Insurance and the Claim Prediction Problem IV

Roadmap for This Chapter

- **Sect. 10.2:** how a Bayesian neural network is set up (priors over weights, likelihood, posterior).
- **Sect. 10.3:** the Laplace approximation used to make that posterior tractable.
- **Sect. 10.4:** the real dataset (3000 Spanish motor policies) and the experimental setup.
- **Sect. 10.5–10.6:** results across six network architectures, and conclusions.

Outline

- 1 10.1 Introduction
- 2 10.2 Bayesian Neural Network Models
- 3 10.3 Laplace Approximation to Bayesian Neural Networks
- 4 Toy Running Example: A Tiny BNN for Motor Claims
- 5 10.4 Numerical Experiments
- 6 10.5 Results and Discussions
- 7 10.6 Conclusion

10.2 What Is a Bayesian Neural Network? – Plain-Language Intuition I

You already know an ordinary (non-Bayesian) feed-forward neural network: data flows forward through layers of neurons, each layer computing a weighted sum of its inputs followed by a non-linear activation, and training finds *one* set of weights that minimises a loss function. A **Bayesian neural network (BNN)** changes only one thing conceptually, but that one change has big consequences.

The Core Idea, in One Sentence

Instead of committing to a single “best” set of weights, put a **prior probability distribution over every weight** in the network, and after seeing the training data, compute a **posterior distribution over the weights** instead of a single fixed value. Feeding an input through this whole distribution of weights (rather than one fixed network) gives you not just a point prediction, but a *distribution of predictions* – genuine, quantified uncertainty.

10.2 What Is a Bayesian Neural Network? – Plain-Language Intuition II

Why This Matters for Insurance Claims

If a model predicts “12% chance of a claim” for two different policyholders, a Bayesian model can additionally tell you *how sure* it is of that 12% for each one. A policy that looks like thousands of others in the training data will get a narrow, confident distribution. A policy with an unusual combination of features (e.g., a very rare vehicle type) will get a wide, uncertain distribution – even if the point estimate is the same 12%. Knowing which is which matters directly for risk management and pricing decisions.

10.2 What Is a Bayesian Neural Network? – Plain-Language Intuition III

The Network Architecture (Eq. 10.1–10.2)

This chapter uses fully connected feed-forward networks. For a network with one hidden layer, the output and hidden-layer computations are:

$$f_k(\mathbf{x}) = b_k + \sum_j v_{jk} h_j(\mathbf{x}), \quad h_j(\mathbf{x}) = \Psi \left(a_j + \sum_i w_{ij} x_i \right).$$

- x_i is the i -th input feature (e.g. driver age, vehicle power); $h_j(\mathbf{x})$ is the activation of the j -th hidden neuron.
- w_{ij} is the weight connecting input i to hidden unit j ; a_j is that hidden unit's bias.
- v_{jk} is the weight connecting hidden unit j to output k (here $k = 1$, a single output: the probability of a claim); b_k is the output bias.
- Ψ is the activation function providing the non-linearity; this chapter uses $\tanh$ throughout for every hidden layer.

In the *ordinary* (non-Bayesian) network, w_{ij}, a_j, v_{jk}, b_k are all just numbers, fixed once training finishes. In a BNN, every one of these is instead treated as a random variable with its own distribution – collect

Python: A Small Feed-Forward Network (the “H” in $P(D | H)$)

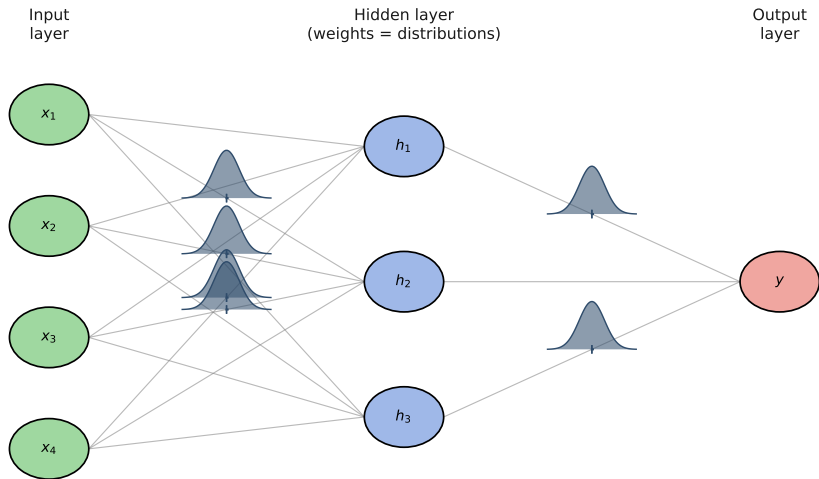
Before adding any Bayesian machinery, we need an ordinary point-estimate network implementing Eqs. 10.1–10.2. This is exactly the object whose weights we will later treat as random variables in Sect. 10.3 – the architecture (H) is fixed by the code below; the *values* of w_1 , w_2 , b_1 , b_2 are what become uncertain in a BNN.

Listing 1: A tiny feed-forward net, matching Eqs. 10.1–10.2

```
1  import numpy as np
2
3  class TinyFeedForwardNet:
4      """One hidden layer, tanh activation, matches Eq. 10.1-10.2."""
5      def __init__(self, n_in, n_hidden, rng):
6          # w_ij: input -> hidden weights; a_j: hidden biases
7          self.w = rng.normal(scale=0.5, size=(n_hidden, n_in))
8          self.a = rng.normal(scale=0.5, size=n_hidden)
9          # v_j: hidden -> output weights; b: output bias
10         self.v = rng.normal(scale=0.5, size=n_hidden)
11         self.b = 0.0
12
13     def forward(self, x):
14         # h_j(x) = tanh(a_j + sum_i w_ij x_i)      -- Eq. 10.2
15         h = np.tanh(self.a + self.w @ x)
16         # f(x) = b + sum_j v_j h_j(x)             -- Eq. 10.1
17         return self.b + self.v @ h
18
19     # H = "1 hidden layer, 5 neurons" is Model 1 in Table 10.1 (137 parameters
```

Illustration: Weights as Distributions, Not Numbers

A Bayesian Neural Network: every weight is a distribution, not a number
(illustrative diagram)



Each little bell curve is one weight $w_{ij} \sim \mathcal{N}(\mu_{ij}, \sigma_{ij}^2)$

narrow bump = confident (low variance); wide bump = uncertain (high variance).

Outline

- 1 10.1 Introduction
- 2 10.2 Bayesian Neural Network Models
- 3 10.3 Laplace Approximation to Bayesian Neural Networks**
- 4 Toy Running Example: A Tiny BNN for Motor Claims
- 5 10.4 Numerical Experiments
- 6 10.5 Results and Discussions
- 7 10.6 Conclusion

10.3 The Laplace Approximation – Plain-Language Intuition I

We now have a posterior $P(\mathbf{w} \mid D, H)$ that we want, but cannot compute exactly. The **Laplace approximation (LA)** is the simplest practical way to approximate it.

The Core Idea, in One Sentence

First find the single best (**MAP**, maximum a posteriori) set of weights $\mathbf{w}_{MP}$ via ordinary training (exactly as you would train any neural network, just with a regularisation term built in). Then approximate the messy true posterior *around that one point* as a **Gaussian** bell curve, using how sharply curved the loss surface is at $\mathbf{w}_{MP}$ – the **Hessian** (matrix of second derivatives) – to set the width of that Gaussian. Sharp curvature (a narrow, steep valley) means a confident, narrow posterior; flat curvature (a wide, shallow valley) means an uncertain, wide posterior.

10.3 The Laplace Approximation – Plain-Language Intuition II

Why a Gaussian Centered at the Mode?

MacKay (1995a) proposed exactly this recipe for Bayesian neural networks. The key practical advantage: $\mathbf{w}_{MP}$ can be obtained with completely standard MAP training of the network (an ordinary training loop with weight decay), and modern second-order optimisation tools make the required curvature (Hessian) estimates cheap to obtain even for networks with hundreds of parameters. This means the Laplace approximation can be bolted onto an *already-trained* deep network, turning point predictions into distributions of predictions with very little extra engineering effort.

10.3 The Laplace Approximation – Plain-Language Intuition III

Setting Up the Exact Posterior (Eq. 10.5–10.6)

Assume the prior on the weights is Gaussian with mean zero and precision α (inverse variance), and let β be the precision of the data-noise term. Then the (still-exact, but intractable) posterior is:

$$P(\mathbf{w} \mid \alpha, \beta, D, H) = \frac{1}{Z_M(\alpha, \beta)} \exp(-(\alpha E_{\mathbf{w}} + \beta E_D)) = \frac{1}{Z_M(\alpha, \beta)} \exp(-M(\mathbf{w}))$$

where $M(\mathbf{w}) := \alpha E_{\mathbf{w}} + \beta E_D$.

- $E_{\mathbf{w}} = \frac{1}{2} \|\mathbf{w}\|^2$ is the **prior kernel**: a penalty growing with the size of the weights (this is exactly an L_2 / weight-decay penalty).
- E_D is the **likelihood kernel**: the data-misfit term (e.g. sum-of-squared errors for regression, or cross-entropy for the claim/no-claim classification task used later in this chapter).
- $\alpha E_{\mathbf{w}} + \beta E_D = M(\mathbf{w})$ is exactly the ordinary, *regularised training loss* of the network. In other words: minimising the familiar training loss *is* finding the mode of this Bayesian posterior.
- $Z_M(\alpha, \beta)$ is just a normalising constant so that the posterior integrates to one.

10.3 The Laplace Approximation – Plain-Language Intuition IV

The Gaussian (Laplace) Approximation Itself (Eq. 10.7–10.8)

MacKay's approximation performs a second-order Taylor expansion of the posterior around its mode $\mathbf{w}_{MP}$:

$$P(\mathbf{w} \mid \alpha, \beta, D, H) \approx \frac{1}{Z'_M(\alpha, \beta)} \exp \left(-M(\mathbf{w}) - \frac{1}{2} W(\alpha, \beta) \right), \quad W(\alpha, \beta) = (\mathbf{w} - \mathbf{w}_{MP})^\top \mathbf{A} (\mathbf{w} - \mathbf{w}_{MP}).$$

- $\mathbf{w}_{MP}$ is the **mode of the posterior** – the MAP estimate, exactly what an ordinary (regularised) training run converges to.
- $\mathbf{A}$ is the matrix of **second derivatives of the negative log posterior** $M(\mathbf{w})$, evaluated at $\mathbf{w}_{MP}$: this is the **Hessian**. It has one row/column per network parameter, so for a network with g parameters, $\mathbf{A}$ is $g \times g$.
- $W(\alpha, \beta)$ is a quadratic form measuring how far a candidate $\mathbf{w}$ is from the mode, *weighted* by the curvature $\mathbf{A}$ in every direction.

10.3 The Laplace Approximation – Plain-Language Intuition V

The Normalising Constant Becomes Tractable (Eq. 10.9–10.10)

$$Z'_M(\alpha, \beta) = \int \exp \left(-M(\mathbf{w}) - \frac{1}{2}(\mathbf{w} - \mathbf{w}_{MP})^\top \mathbf{A}(\mathbf{w} - \mathbf{w}_{MP}) \right) d\mathbf{w} = \exp(-M(\mathbf{w})) (2\pi)^{-g/2} \det(\mathbf{A})^{-\frac{1}{2}}$$

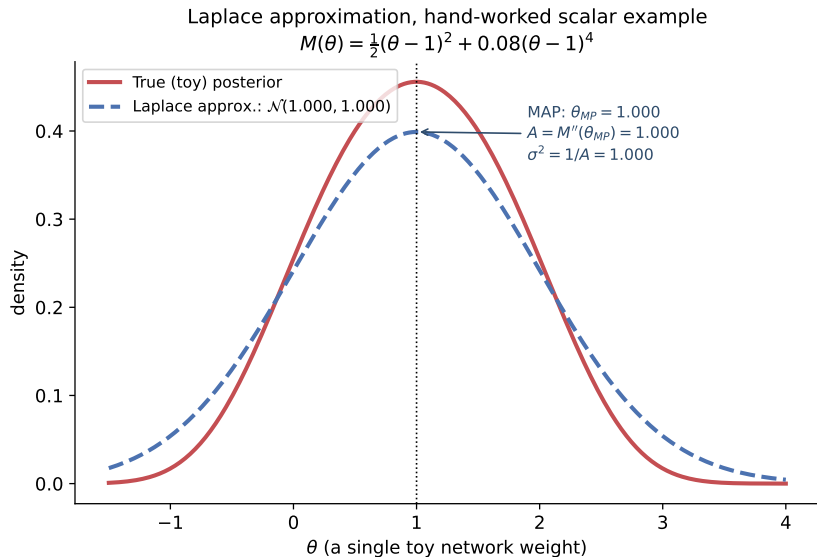
Here g is the number of parameters in the network, and $\det(\mathbf{A})$ is the determinant of the Hessian. This is precisely the normalising constant of a multivariate Gaussian – confirming that the Laplace approximation is exactly $\mathbf{w} \mid D \sim \mathcal{N}(\mathbf{w}_{MP}, \mathbf{A}^{-1})$. Because this expression is now closed-form, both the evidence $P(D \mid H)$ and the optimal prior/noise precisions α, β (found by alternating gradient descent on the weights with evidence maximisation over α, β) become computationally cheap to evaluate. This chapter uses the **Laplace** package in Python (Daxberger et al., 2021) to do exactly this.

10.3 The Laplace Approximation – Plain-Language Intuition VI

The Laplace Approximation, Step by Step

- 1 **Find the MAP estimate.** Train the network as usual, minimising $M(\mathbf{w}) = \alpha E_{\mathbf{w}} + \beta E_D$ (an ordinary loss plus a weight-decay penalty). The result is $\mathbf{w}_{MP}$, a single best set of weights – exactly what you would get from standard (non-Bayesian) training.
- 2 **Compute (or approximate) the Hessian.** Evaluate $\mathbf{A} = \nabla_{\mathbf{w}}^2 M(\mathbf{w})|_{\mathbf{w}=\mathbf{w}_{MP}}$, the matrix of second derivatives of the negative log posterior at the MAP estimate. This measures how sharply the loss curves in every direction of weight space around $\mathbf{w}_{MP}$.
- 3 **Set the posterior covariance to the inverse Hessian.** Approximate $P(\mathbf{w} \mid D, H) \approx \mathcal{N}(\mathbf{w}_{MP}, \mathbf{A}^{-1})$. Directions of high curvature (large entries of $\mathbf{A}$) get small posterior variance (confident); directions of low curvature (small entries of $\mathbf{A}$) get large posterior variance (uncertain).
- 4 **Sample or propagate uncertainty through the network.** To get predictive uncertainty for a new input $\mathbf{x}^*$, either (a) draw many weight samples $\mathbf{w}^{(s)} \sim \mathcal{N}(\mathbf{w}_{MP}, \mathbf{A}^{-1})$ and run the network forward for each one, building up a distribution of predictions, or (b) linearise the network output around $\mathbf{w}_{MP}$ and propagate the Gaussian directly (the “delta method”): $\text{Var}[f(\mathbf{x}^*)] \approx \mathbf{J}(\mathbf{x}^*) \mathbf{A}^{-1} \mathbf{J}(\mathbf{x}^*)^\top$, where $\mathbf{J}(\mathbf{x}^*)$ is the gradient of the network output with respect to $\mathbf{w}$ at $\mathbf{x}^*$.

Illustration: Laplace Approximation of a Non-Gaussian Posterior



Python: Laplace Approximation Step by Step (Toy Example)

The code below performs the four Laplace-approximation steps on a tiny toy network by hand: find the MAP estimate by minimising the penalised loss, numerically approximate the Hessian at that point, invert it to get the posterior covariance, and use the resulting Gaussian to get a predictive standard deviation. This mirrors exactly what a library such as `laplace-torch` automates for a full-sized network.

Listing 2: MAP, Hessian, and predictive uncertainty by hand

```
1 import numpy as np
2 from scipy.optimize import minimize
3
4 def neg_log_posterior(theta, x, y, alpha, beta, forward):
5     #  $M(w) = \alpha E_w + \beta E_D$  (Eq. 10.6)
6     resid = y - forward(theta, x)
7     return 0.5*alpha*np.sum(theta**2) + 0.5*beta*np.sum(resid**2)
8
9 # Step 1: find  $w_{MP}$  by minimising  $M(w)$  -- ordinary MAP training
10 result = minimize(neg_log_posterior, theta0,
11                  args=(x_train, y_train, alpha, beta, forward),
12                  method="BFGS", jac=grad_neg_log_posterior)
13 theta_map = result.x
14
15 # Step 2: numerically approximate the Hessian  $A$  at  $\theta_{map}$ 
16 #         (finite-difference the analytic gradient)
17 A = numerical_hessian(theta_map, x_train, y_train, alpha, beta)
```

Outline

- 1 10.1 Introduction
- 2 10.2 Bayesian Neural Network Models
- 3 10.3 Laplace Approximation to Bayesian Neural Networks
- 4 Toy Running Example: A Tiny BNN for Motor Claims**
- 5 10.4 Numerical Experiments
- 6 10.5 Results and Discussions
- 7 10.6 Conclusion

A Toy Running Example: Driver Age vs. Claim Amount I

To make everything concrete, consider a miniature version of a motor-insurance-claims regression problem: 6 synthetic (driver age, claim amount) pairs, loosely mimicking the well-known actuarial pattern that both very young and older drivers tend to have higher claim costs than middle-aged drivers (a U-shaped risk profile).

This toy dataset and toy network are entirely our own illustrative simulation, built only to demonstrate the mechanics of Sects. 10.2–10.3. They are not the book's real data or real results – those are summarised qualitatively in Sects. 10.4–10.5.

The Toy Data

Driver age (yrs)	22	28	35	45	55	65
Claim amount (R'000)	4.8	3.1	2.0	2.4	3.4	5.1

A Toy Running Example: Driver Age vs. Claim Amount II

The Toy Network

We fit a tiny network with two $\tanh$ hidden units plus a linear term:

$$f(x; \mathbf{w}) = v_1 \tanh(w_1 x + b_1) + v_2 \tanh(w_2 x + b_2) + m x + c,$$

an 8-parameter model ($\mathbf{w} = (w_1, b_1, v_1, w_2, b_2, v_2, m, c)$), with a Gaussian prior of precision $\alpha = 0.3$ on every parameter and a Gaussian-noise likelihood of precision $\beta = 8$ (i.e. an assumed observation noise standard deviation of $1/\sqrt{8} \approx 0.35$, in R'000). We follow exactly the four Laplace steps of Sect. 10.3.

A Toy Running Example: Driver Age vs. Claim Amount III

Step 1 – Step 3 Applied to the Toy Data

- **Step 1 (MAP):** minimising $M(\mathbf{w})$ by gradient-based optimisation gives $w_1=-2.07$, $b_1=-1.90$, $v_1=2.35$, $w_2=-1.19$, $b_2=1.02$, $v_2=-1.04$, $m=0.88$, $c=5.13$. This is exactly what an ordinary (regularised) training run of this tiny network would converge to.
- **Step 2 (Hessian):** we numerically approximate the 8×8 Hessian $\mathbf{A}$ of $M(\mathbf{w})$ at this MAP point by central-differencing the analytic gradient. Checking its eigenvalues confirms $\mathbf{A}$ is positive definite – confirming this MAP point is a genuine local minimum of the loss (a valid prerequisite for the Laplace approximation).
- **Step 3 (posterior covariance):** $\Sigma = \mathbf{A}^{-1}$. Some parameters (like c , the output bias) end up tightly constrained by the data; others (like the input-side hidden weights) are left with more residual uncertainty, since only 6 data points constrain an 8-parameter model.

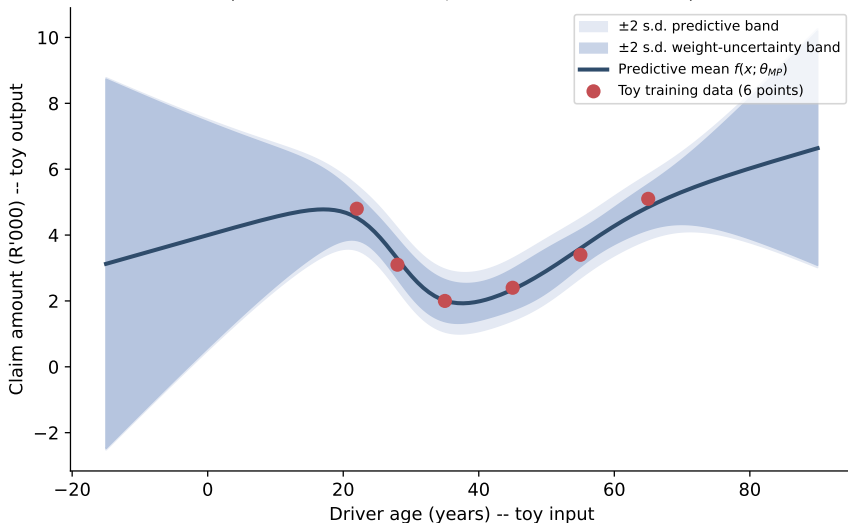
A Toy Running Example: Driver Age vs. Claim Amount IV

Step 4: Propagating Uncertainty to Predictions

For any query age x^* , the predictive variance is $\text{Var}[f(x^*)] \approx \mathbf{J}(x^*)^\top \boldsymbol{\Sigma} \mathbf{J}(x^*)$, where $\mathbf{J}(x^*)$ is the (analytic) gradient of $f(x^*; \mathbf{w})$ with respect to every one of the 8 parameters at $\mathbf{w}_{MP}$. The total predictive standard deviation additionally adds the observation noise, $\sqrt{\text{Var}[f(x^*)] + 1/\beta}$.

Toy Result: Predictive Mean and Uncertainty Band

Toy 1-D BNN regression via the Laplace approximation
(illustrative simulation, not the book's real data)



Outline

- 1 10.1 Introduction
- 2 10.2 Bayesian Neural Network Models
- 3 10.3 Laplace Approximation to Bayesian Neural Networks
- 4 Toy Running Example: A Tiny BNN for Motor Claims
- 5 10.4 Numerical Experiments**
- 6 10.5 Results and Discussions
- 7 10.6 Conclusion

10.4.1 Data Description I

Having built the toy intuition, we return to the book's real numerical experiments. All numbers in this section and the next are as reported in the chapter – not toy numbers.

The Dataset

3000 records of a non-life motor insurance portfolio from a **Spanish insurance company** (Segura-Gisbert et al., 2024). The target variable is **binary**: whether or not the policy had a claim in the given year. Only **18% of instances were claims** – the dataset is **highly imbalanced**. All features are normalised with a **min-max scaler** before being used.

10.4.1 Data Description II

The 27 Input Features (Grouped)

Policy administration: ID, contract start/renewal/next-renewal dates, distribution channel (agent vs. broker), seniority, policies in force, max policies/products ever held, lapse count and date, payment method, premium.

Claims behaviour: R_Claims history – the proportion of claims relative to total years insured, excluding the current year.

Driver & policy risk: date of birth, driving-licence date, type of risk (motorcycle/van/automobile/agricultural), area (rural/urban), second driver flag.

Vehicle characteristics: year of registration, power, cylinder capacity, market value, number of doors, fuel type (petrol/diesel), length, weight.

10.4.1 Data Description III

Why the ARD Prior on These Features Matters

An Automatic Relevance Determination (ARD) prior places a *separate* precision hyperparameter on the group of weights entering from each input feature. Features that are not useful for prediction get shrunk strongly toward zero (their effective weight variance collapses); useful features keep meaningful weight variance. This is what allows the chapter to *rank* the 27 input features by importance in Sect. 10.5, as a direct byproduct of Bayesian training – not a separate post-hoc analysis step.

10.4.2 Experiment Setup and Performance Metrics I

Training Setup

- A **standard Gaussian** distribution is used as the prior for the weights.
- The Laplace approximation algorithm is run for **150 iterations**.
- All computations are performed in Python using the **Laplace PyTorch package** (Daxberger et al., 2021) – the same practical tool introduced conceptually in Sect. 10.3.
- The **log evidence** is computed on the training data for each of the six architectures (Table 10.1 of Sect. 10.2), used for model selection: the higher the log evidence, the better the model is considered to be, *without* needing to look at held-out data.

10.4.2 Experiment Setup and Performance Metrics II

Performance Metrics on Unseen (Held-Out) Data

- **ROC (Receiver Operating Characteristic) curve:** plots the true positive rate (correctly flagged claims) against the false positive rate (false alarms) as the decision threshold varies.
- **AUC (Area Under the ROC Curve):** summarises the ROC curve in a single number between 0 and 1; measures the average rate of correct ranking between a random claim and a random non-claim. AUC is particularly well suited here because the correct relative cost of a false positive vs. a false negative is not known in advance – exactly the situation with this imbalanced (18% claims) dataset.
- **Accuracy:** the overall fraction of correctly classified policies on unseen data.

Python: Bayesian Neural Networks via the laplace-torch Package

This is (schematically) the actual workflow the book uses: train a point-estimate network exactly as usual, then wrap it with the Laplace package to obtain the posterior over weights, all without touching the network's training code.

Listing 3: Schematic workflow following Daxberger et al. (2021)

```
1 import torch, torch.nn as nn
2 from laplace import Laplace
3
4 # 1) Ordinary MAP training: build & train the point-estimate network,
5 #    e.g. Model 5 -- 2 hidden layers, 10 tanh neurons each (372 params)
6 model = nn.Sequential(
7     nn.Linear(27, 10), nn.Tanh(),
8     nn.Linear(10, 10), nn.Tanh(),
9     nn.Linear(10, 1),
10 )
11 # ... standard training loop with binary cross-entropy loss on
12 #    whether the policy had a claim ...
13
14 # 2) Wrap the trained network with a Laplace approximation
15 la = Laplace(model, likelihood="classification",
16             subset_of_weights="all", hessian_structure="full")
17 la.fit(train_loader) # Step 2: estimate the Hessian A
18 la.optimize_prior_precision() # optimise alpha, beta by max evidence
19
20 print("Log evidence:", la.log_marginal_likelihood().item())
21
```

Outline

- 1 10.1 Introduction
- 2 10.2 Bayesian Neural Network Models
- 3 10.3 Laplace Approximation to Bayesian Neural Networks
- 4 Toy Running Example: A Tiny BNN for Motor Claims
- 5 10.4 Numerical Experiments
- 6 10.5 Results and Discussions**
- 7 10.6 Conclusion

10.5 Results: Model Comparison via the Evidence I

Table 10.1 – Log Evidence, AUC, and Accuracy Across Models

Metric	M1	M2	M3	M4	M5	M6
Parameters	137	272	1352	162	372	472
Log evidence	-379.27	-375.75	-390.80	-377.90	-336.27	-363.50
AUC	0.7779	0.7828	0.8035	0.7561	0.8407	0.7680
Accuracy	0.8908	0.8915	0.8962	0.8915	0.9131	0.8938

Model 5 (two hidden layers, 10 neurons each, 372 parameters) has both the **highest log evidence** and the **best AUC and accuracy** on unseen data – all three metrics agree. Notably, Model 3 (the largest, with 1352 parameters) has the *worst* log evidence: the evidence metric penalises unnecessarily complex models, favouring parsimony rather than raw parameter count.

10.5 Results: Model Comparison via the Evidence II

Table 10.2 – Pairwise Log Bayes Factors

	M1	M2	M3	M4	M5	M6
M1	x	3.51	−11.53	1.36	42.99	15.76
M2		x	−15.04	−2.15	39.47	12.25
M3			x	12.89	54.52	27.30
M4				x	41.63	14.40
M5					x	−27.22

Reading the table: entries compare the column model against the row model (e.g. the Bayes factor for Model 5 vs. Model 2 is 39.47, strongly favouring Model 5). **Model 6 has the highest support against every other model except Model 5**, which beats every model including Model 6. All pairwise comparisons show a significant difference according to Jeffreys' scale for interpreting Bayes factors. The overall pattern: **deeper architectures are preferred to wider-but-shallow ones.**

Figure 10.2 in the Book: Predictive Performance and Evidence Convergence

Panel (a): ROC Curves (Described Qualitatively)

ROC curves for all six models on unseen data; **Model 5** sits closest to the top-left corner (AUC = 0.8407), visibly above the other five curves, all of which comfortably beat the “no skill” diagonal (AUC = 0.5). The ordering of the curves matches the AUC column of Table 10.1 exactly.

Panel (b): Marginal-Likelihood Convergence

The evidence (marginal likelihood) value plotted across all 150 training iterations, for every one of the six models. Every curve flattens out by the end of training – confirming that the Laplace fitting procedure has converged for every architecture compared, so the final log-evidence values in Table 10.1 are trustworthy for model comparison rather than artefacts of early stopping.

Why This Matters

Seeing *both* panels together is important: a model could have high AUC purely by chance, or its evidence value could be unreliable if training had not converged. Here, the AUC ranking (data-driven, held-out) and the evidence ranking (model-driven, computed only on training data) agree, and both

Figure 10.3 and Table 10.3: Which Features Matter Most? I

How Feature Importance Falls Out of the ARD Prior

Recall from Sect. 10.4.1 that the ARD prior gives each input feature its own precision hyperparameter. The book's Fig. 10.3 plots the **mean posterior variance** of the weights associated with each input feature, for every one of the six models: a taller bar means the model has left more posterior uncertainty (equivalently, allowed more weight magnitude) for that feature, meaning the ARD mechanism judged it *more* important, not less.

Table 10.3 – Top-6 Ranked Features by Model

Rank	M1	M2	M3	M4	M5	M6
1	R Claims hist.	R Claims hist.	R Claims hist.	R Claims hist.	R Claims hist.	R Claims hist.
2	Type fuel	Type fuel	Policies in force	Type fuel	Type fuel	Date start contract
3	Lapse	Date next renewal	Lapse	Policies in force	Type risk	Type fuel
4	Power	Max products	Payment	Max policies	N doors	Cylinder capacity
5	Date last renewal	Power	Max products	N doors	Cylinder capacity	Policies in force
6	Max policies	Date last renewal	Date next renewal	Seniority	Lapse	Power

Figure 10.3 and Table 10.3: Which Features Matter Most? II

The Headline Finding

Every single model agrees: `R_Claims` history (the policyholder's historical claim frequency) is by far the **single most important predictor**, ranked #1 across all six architectures. This is followed consistently by **vehicle-related features** (fuel type, power, cylinder capacity) and by **how many policies/products** the client holds with the insurer – a broadly consistent, robust picture of what drives claim risk in this portfolio.

Outline

- 1 10.1 Introduction
- 2 10.2 Bayesian Neural Network Models
- 3 10.3 Laplace Approximation to Bayesian Neural Networks
- 4 Toy Running Example: A Tiny BNN for Motor Claims
- 5 10.4 Numerical Experiments
- 6 10.5 Results and Discussions
- 7 10.6 Conclusion

10.6 Conclusion I

What This Chapter Achieved

This chapter presented a **first-in-literature Bayesian framework** for modelling motor insurance claims: Bayesian neural networks with Automatic Relevance Determination priors, trained via the **Laplace approximation**. This framework delivers two things simultaneously that a point-estimate neural network cannot: (1) a principled way to **select between competing architectures** using the Bayesian evidence, and (2) a **ranking of which input features matter most** for the prediction task, as a natural byproduct of the ARD prior.

10.6 Conclusion II

Key Results

- The Bayesian evidence metric from the Laplace approximation reliably selects the model (Model 5: two hidden layers, 10 neurons each) that *also* performs best on held-out AUC and accuracy – model selection via evidence and via held-out performance agree.
- **Deeper** network architectures are preferred over **wider-but-shallow** ones for this dataset.
- The most critical variable across every architecture is **claim history**, followed by vehicle-related features (fuel type, power) and by the number of policies/products the client holds with the insurer.

10.6 Conclusion III

Limitations and Future Work (as noted in the chapter)

- The chapter used the Laplace approximation rather than MCMC; MCMC methods are guaranteed to converge to the true posterior (given enough iterations) and could be explored as an extension, at greater computational cost.
- The chapter modelled only *whether* a claim occurs (a binary classification problem). Future work could instead model the *absolute value* of claims, decomposing the problem into claim **frequency** and claim **severity** – much closer to how actuaries traditionally frame the pricing problem.

Looking Ahead

The next chapter turns to a very different application of Bayesian computation in quantitative finance: **shadow and adaptive Hamiltonian Monte Carlo methods** are used to calibrate the **Nelson–Siegel yield curve model**, again with the goal of obtaining not just point estimates of the calibrated parameters, but full uncertainty quantification over both the parameters and the model's yield-curve predictions.